FounderConsole Feature Implementation Prompt for Replit

📋 PROJECT CONTEXT

You are building enhancements for **FounderConsole** — an AI-powered financial intelligence platform for startups. The platform currently has:

- Monte Carlo simulation engine with P10/P50/P90 confidence bands
- AI Copilot with multi-LLM routing (GPT-4, Claude, Gemini)
- Truth Scan for data validation
- Cap table management
- Scenario modeling and comparison
- Decision Journal and Goal Tracking

Tech Stack: Modern React (Next.js), TypeScript, Tailwind CSS, shadcn/ui components

---

🎯 FEATURES TO IMPLEMENT

Build these 4 high-priority features with production-quality code:

FEATURE 1: Board Deck Export Generator

User Story: As a founder, I want to export my simulations and metrics into investor-ready slides so I can prepare for board meetings in minutes, not hours.

Requirements:

1. **Export Button Component**
   - Location: Top-right of Dashboard, Scenarios, and Decisions pages
   - Icon: Download/Presentation icon
   - Dropdown menu with options: “Monthly Update,” “Fundraising Prep,” “Scenario Analysis,” “Custom Report”

2. **Template System**

```
interface BoardTemplate {
  id: string;
  name: string;
  description: string;
  sections: TemplateSection[];
}

interface TemplateSection {
  type: 'metrics' | 'chart' | 'simulation' | 'narrative' | 'comparison';
  title: string;
  dataSource: string;
  aiNarrative?: boolean;
}
```

3. **Pre-built Templates:**

   A. Monthly Update Template:

- Slide 1: Executive Summary (MRR, growth %, runway, burn)
- Slide 2: Key Metrics Dashboard (ARR, customers, LTV:CAC, churn)
- Slide 3: Cash & Runway (cash balance, runway trend, 6-month projection)
- Slide 4: Goals Progress (tracked goals with % complete)
- Slide 5: Key Decisions Made (from Decision Journal)
- Slide 6: Upcoming Priorities (next 30 days)

B. Fundraising Prep Template:

- Slide 1: Company Snapshot (ARR, growth, customers, team size)
- Slide 2: Unit Economics (LTV, CAC, payback, margins with benchmarks)
- Slide 3: Growth Trajectory (MRR chart with projections)
- Slide 4: Runway Analysis (current + post-raise scenarios)
- Slide 5: Market Opportunity (TAM/SAM/SOM with AI research)
- Slide 6: Use of Funds (hiring plan, burn projection)

C. Scenario Analysis Template:

- Slide 1: Decision Context (what we're evaluating)
- Slide 2: Scenario Comparison Table (side-by-side metrics)
- Slide 3: Monte Carlo Results (P10/P50/P90 bands)
- Slide 4: Sensitivity Analysis (which variables matter most)
- Slide 5: AI Recommendation (with reasoning)
- Slide 6: Next Steps

4. **Export Formats:**
   - PDF (primary) — use html2pdf.js or puppeteer
   - Google Slides (via Google Apps Script API)
   - PowerPoint (via pptxgenjs library)
5. **AI Narrative Generation:**
   - For each section with `aiNarrative: true`, generate 2-3 sentence summary
   - Example: "MRR grew 12% MoM to $43.9K, driven by 3 new enterprise customers. At current trajectory, we'll hit $50K MRR by June, 3 weeks ahead of plan."
6. **Visual Design:**
   - Clean, professional slides with FounderConsole branding
   - Charts rendered as images in exports
   - Consistent color scheme (use existing brand colors)

File Structure:

```
app/components/board-export/
  ├── ExportButton.tsx
  ├── TemplateSelector.tsx
  ├── PreviewModal.tsx
  ├── templates/
  │   ├── monthlyUpdate.ts
  │   ├── fundraisingPrep.ts
  │   ├── scenarioAnalysis.ts
  │   └── index.ts
  ├── generators/
```

```
│   ├── pdfGenerator.ts
│   ├── slidesGenerator.ts
│   └── narrativeGenerator.ts
└── hooks/
    └── useBoardExport.ts
```

FEATURE 2: Hiring Planner with Burn Impact

User Story: As a founder, I want to model hiring plans and see exact runway impact so I can make informed team growth decisions.

Requirements:

1. **Hiring Planner Page** (`/hiring-planner`)
   - New sidebar navigation item between "Simulate" and "Decisions"
   - Page title: "Hiring Planner" with subtitle "Model team growth and runway impact"
2. **Role Library:**

```typescript
interface Role {
  id: string;
  title: string;
  department: 'engineering' | 'sales' | 'marketing' | 'product' | 'operations' |
  ↪  'customer-success';
  baseSalary: {
    low: number;
    mid: number;
    high: number;
  };
  location: 'sf' | 'ny' | 'remote-us' | 'international';
  additionalCosts: {
    benefits: number; // % of salary
    equipment: number;
    onboarding: number;
  };
}
```

   Pre-populate with 20 common startup roles:
   - Engineering: Senior Engineer, Staff Engineer, Engineering Manager, CTO
   - Sales: SDR, AE, Sales Manager, VP Sales
   - Marketing: Marketing Manager, Content Marketer, Growth Marketer
   - Product: Product Manager, Product Designer
   - Operations: Office Manager, HR Manager, Finance Manager
   - Customer Success: CSM, Support Rep
3. **Hiring Plan Builder:**

```typescript
interface HiringPlan {
  id: string;
  name: string;
```

```
  hires: PlannedHire[];
  startDate: Date;
  totalCost: number;
  runwayImpact: number;
}

interface PlannedHire {
  roleId: string;
  count: number;
  startMonth: number; // 0 = this month, 1 = next month, etc.
  salary: number;
  location: string;
}
```

4. **UI Components:**

   A. Role Selector:

   - Searchable dropdown of roles
   - Show: Title, Department, Salary Range (based on location)
   - Filter by department

   B. Hiring Timeline:

   - Visual timeline showing hires by month (next 12 months)
   - Drag to reorder hires between months
   - Show headcount per month

   C. Cost Breakdown:

   - Monthly cost increase (salary + benefits + other)
   - One-time costs (equipment, onboarding, recruiting fees)
   - Total first-year cost

   D. Runway Impact Panel:

   - Current runway: X months
   - After hiring plan: Y months
   - Difference: -Z months
   - Visual: Side-by-side runway bars

5. **Scenario Integration:**

   - "Run Simulation" button converts hiring plan to scenario
   - Runs Monte Carlo with new burn rate
   - Shows P10/P50/P90 runway with hiring plan

6. **Quick Actions:**

   - "Hire 3 Engineers in Q2" — one-click template
   - "Double Sales Team" — adds SDRs and AEs with typical ratios
   - "Hiring Freeze" — removes all planned hires

7. **Comparison Mode:**

   - Compare 2-3 hiring plans side-by-side

- Show: headcount, monthly burn increase, runway impact, cost per role

File Structure:

```
app/hiring-planner/
  ├── page.tsx
  ├── components/
  │   ├── RoleSelector.tsx
  │   ├── HiringTimeline.tsx
  │   ├── CostBreakdown.tsx
  │   ├── RunwayImpact.tsx
  │   ├── HiringPlanCard.tsx
  │   └── ComparisonView.tsx
  ├── data/
  │   └── roles.ts
  ├── hooks/
  │   └── useHiringPlan.ts
  └── types/
      └── hiring.ts
```

FEATURE 3: Fundraising Readiness Score & Timeline

User Story: As a founder, I want to know when I'm ready to raise and what to fix first so I can time my fundraise optimally.

Requirements:

1. **Fundraising Page Enhancement** (`/fundraising`)
   - Add new section: "Readiness Assessment" above existing cap table
2. **Readiness Score Algorithm:**

```typescript
interface ReadinessScore {
  overall: number; // 0-100
  breakdown: {
    runway: { score: number; weight: 0.25 };
    growth: { score: number; weight: 0.25 };
    unitEconomics: { score: number; weight: 0.20 };
    marketTiming: { score: number; weight: 0.15 };
    narrative: { score: number; weight: 0.15 };
  };
  status: 'not-ready' | 'getting-close' | 'ready' | 'optimal';
  recommendations: Recommendation[];
}

interface Recommendation {
  category: string;
  issue: string;
  impact: 'high' | 'medium' | 'low';
  action: string;
  targetMetric: string;
  currentValue: number;
```

```
  targetValue: number;
}
```

3. **Scoring Criteria:**

Metric	Not Ready (<40)	Getting Close (40-70)	Ready (70-85)	Optimal (85+)
Runway	<9 months	9-15 months	15-24 months	24+ months
MRR Growth	<5% MoM	5-10% MoM	10-20% MoM	20%+ MoM
LTV:CAC	<2x	2-3x	3-4x	4x+
Gross Margin	<60%	60-70%	70-80%	80%+
NRR	<100%	100-110%	110-120%	120%+

4. **UI Components:**

   A. Readiness Score Card:

   - Large circular progress indicator showing overall score (0-100)
   - Color-coded: Red (<40), Yellow (40-70), Light Green (70-85), Green (85+)
   - Status badge: "Not Ready" / "Getting Close" / "Ready" / "Optimal"

   B. Breakdown Radar Chart:

   - 5 axes: Runway, Growth, Unit Economics, Market Timing, Narrative
   - Shows current vs target
   - Highlights weakest areas

   C. Recommendations List:

   - Sorted by impact (high → low)
   - Each item shows:
     - Issue (e.g., "LTV:CAC is 2.5x, below 3x threshold")
     - Action (e.g., "Improve retention or reduce CAC")
     - Impact on score if fixed
     - "Run Scenario" button to model the fix

   D. Optimal Raise Window:

   - "Based on your trajectory, optimal raise window: June-August 2026"
   - Shows: current score trajectory, projected score by month
   - "If you fix [top recommendation], raise window moves to: April-June 2026"

5. **Investor Matching:**

```
interface InvestorMatch {
  firm: string;
  stage: string;
  checkSize: string;
  sectorFocus: string[];
  matchScore: number;
  reason: string;
  warmIntro?: string;
```

```
}
```

- Show 5-10 recommended firms based on stage/sector
- Match score based on portfolio fit
- Link to firm website + partner profiles

6. **Fundraising Timeline:**
   - Visual timeline: Prep → Outreach → Meetings → Term Sheets → Close
   - Current position indicator
   - Estimated duration: "12-16 weeks typical for your stage"
7. **One-Pager Generator:**
   - Auto-generate investment memo using company data
   - Sections: Problem, Solution, Traction, Market, Team, Financials, Ask
   - Export to PDF

File Structure:

```
app/fundraising/
  ├── page.tsx (enhance existing)
  ├── components/
  │   ├── ReadinessScore.tsx
  │   ├── ReadinessBreakdown.tsx
  │   ├── RecommendationsList.tsx
  │   ├── RaiseWindow.tsx
  │   ├── InvestorMatching.tsx
  │   ├── FundraisingTimeline.tsx
  │   └── OnePagerGenerator.tsx
  ├── lib/
  │   └── readinessScoring.ts
  └── data/
      └── investors.ts
```

FEATURE 4: Smart Alerts & Anomalies

User Story: As a founder, I want to be notified when key metrics change significantly so I can catch problems early and celebrate wins.

Requirements:

1. **Alert System Architecture:**

```
interface Alert {
  id: string;
  type: 'burn-spike' | 'mrr-drop' | 'churn-spike' | 'runway-warning' | 'goal-milestone' |
  ↪  'anomaly-detected' | 'benchmark-exceeded';
  severity: 'critical' | 'warning' | 'info' | 'success';
  title: string;
  message: string;
  metric: string;
  currentValue: number;
```

```
  previousValue: number;
  changePercent: number;
  timestamp: Date;
  acknowledged: boolean;
  suggestedAction?: string;
}

interface AlertRule {
  id: string;
  metric: string;
  condition: 'above' | 'below' | 'change-percent' | 'change-absolute';
  threshold: number;
  lookbackDays: number;
  enabled: boolean;
  channels: ('email' | 'slack' | 'in-app')[];
}
```

2. **Pre-configured Alert Rules:**

Alert	Condition	Severity	Message
Burn Spike	Burn increased >15% WoW	Critical	“Burn rate increased 18% this week. Investigate immediately.”
MRR Drop	MRR decreased >5% MoM	Critical	“MRR dropped 7% this month. Churn spike detected.”
Churn Spike	Churn increased >50%	Warning	“Churn jumped from 2% to 3.5%. Review exit surveys.”
Runway Warning	Runway <12 months	Critical	“Runway dropped to 10 months. Time to plan fundraise or cut costs.”
Runway Caution	Runway <18 months	Warning	“Runway at 16 months. Consider scenario planning.”
Growth Slowdown	Growth rate dropped >30%	Warning	“Growth slowed from 15% to 8% MoM. What’ s changed?”
Goal Milestone	Goal 80%+ complete	Success	“You’ re 85% to $50K MRR goal —almost there!”
Benchmark Exceeded	Metric >75th percentile	Success	“Your LTV:CAC is now top 25% for Seed SaaS!”

3. **Anomaly Detection:**

- Use Z-score algorithm (you already have this in Truth Scan)
- Flag metrics that are 2+ standard deviations from historical average
- Example: "CAC spiked to $2.5K (normally $1.5K). Investigate channel performance."

4. **Alert Center UI** (`/alerts`)
   - New sidebar item: "Alerts" with badge showing unread count
   - List view: All alerts sorted by date (newest first)
   - Filter by: severity, type, date range, acknowledged status
   - Actions: Acknowledge, Dismiss, "Run Simulation" (for actionable alerts)
5. **Alert Detail View:**
   - Full context: chart showing metric over time
   - Comparison to benchmarks
   - Suggested actions with buttons:
     - "Run Scenario" → opens simulation with suggested parameters
     - "View Details" → jumps to relevant dashboard section
     - "Mark Resolved" → for alerts that have been addressed
6. **Notification Channels:**

   A. In-App:
   - Bell icon in header with unread badge
   - Dropdown showing recent alerts
   - Red dot on sidebar "Alerts" item

   B. Email:
   - Weekly digest: "Your FounderConsole Weekly Briefing"
   - Immediate alerts for critical issues
   - Template with company branding

   C. Slack Integration:
   - OAuth connection to Slack workspace
   - Post alerts to designated channel
   - Include "View in FounderConsole" button
7. **Weekly Briefing Email:**
   - Sent every Monday at 9am
   - Subject: "Your Weekly Founder Briefing — [Company Name]"
   - Content:
     - Executive Summary (MRR, burn, runway changes)
     - Key Metric Changes (up/down with %)
     - This Week's Focus (goals due this week)
     - Recommended Simulation (based on recent changes)
     - Benchmark Comparison (how you stack up)
8. **Alert Settings:**
   - Enable/disable each alert type
   - Customize thresholds
   - Choose notification channels per alert type

- Quiet hours (don't send Slack alerts 10pm-8am)

File Structure:

```
app/alerts/
  ├── page.tsx
  ├── components/
  │   ├── AlertList.tsx
  │   ├── AlertCard.tsx
  │   ├── AlertDetail.tsx
  │   ├── AlertSettings.tsx
  │   ├── NotificationPreferences.tsx
  │   └── SlackIntegration.tsx
  ├── hooks/
  │   ├── useAlerts.ts
  │   └── useAlertRules.ts
  └── lib/
      ├── alertEngine.ts
      └── anomalyDetection.ts

app/api/alerts/
  ├── route.ts (CRUD alerts)
  ├── rules/route.ts (manage rules)
  └── notify/route.ts (send notifications)

app/api/integrations/slack/
  ├── auth/route.ts (OAuth)
  └── webhook/route.ts (incoming/outgoing)
```

🎨 DESIGN SYSTEM

Use the existing FounderConsole design system:

Colors:

- Primary: `#0F172A` (slate-900) — headers, primary buttons
- Secondary: `#3B82F6` (blue-500) — links, active states
- Success: `#10B981` (emerald-500) — positive metrics, success states
- Warning: `#F59E0B` (amber-500) — caution, medium alerts
- Danger: `#EF4444` (red-500) — critical alerts, negative trends
- Background: `#FFFFFF` (white) — main background
- Surface: `#F8FAFC` (slate-50) — cards, sections

Typography:

- Headings: Inter, font-weight 600-700
- Body: Inter, font-weight 400-500
- Monospace: JetBrains Mono for numbers

Components:

- Use existing shadcn/ui components
- Cards: rounded-xl, shadow-sm, border border-slate-200
- Buttons: rounded-lg, consistent padding (px-4 py-2)
- Charts: Recharts library (consistent with existing)

🔧 TECHNICAL REQUIREMENTS

Dependencies to Install:

```
# PDF/Export
npm install html2pdf.js pptxgenjs

# Charts (if not already installed)
npm install recharts

# Date handling
npm install date-fns

# Slack SDK
npm install @slack/web-api

# Email (if not already set up)
npm install @sendgrid/mail
```

Database Schema Additions:

```
-- Hiring Plans
CREATE TABLE hiring_plans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies(id),
  name VARCHAR(255),
  hires JSONB, -- array of PlannedHire
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Alerts
CREATE TABLE alerts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies(id),
  type VARCHAR(50),
  severity VARCHAR(20),
  title VARCHAR(255),
  message TEXT,
  metric VARCHAR(100),
  current_value DECIMAL,
  previous_value DECIMAL,
  change_percent DECIMAL,
```

```
  acknowledged BOOLEAN DEFAULT FALSE,
  suggested_action TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Alert Rules
CREATE TABLE alert_rules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies(id),
  metric VARCHAR(100),
  condition VARCHAR(50),
  threshold DECIMAL,
  lookback_days INTEGER,
  enabled BOOLEAN DEFAULT TRUE,
  channels JSONB, -- array of channels
  created_at TIMESTAMP DEFAULT NOW()
);

-- Integrations
CREATE TABLE integrations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies(id),
  type VARCHAR(50), -- 'slack', 'email', etc.
  config JSONB, -- OAuth tokens, settings
  connected_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

API Endpoints:

```
// Board Export
POST /api/export/board-deck
Body: { templateId: string, format: 'pdf' | 'slides' | 'pptx', sections: string[] }

// Hiring Planner
GET /api/hiring-plans
POST /api/hiring-plans
PUT /api/hiring-plans/:id
DELETE /api/hiring-plans/:id
POST /api/hiring-plans/:id/simulate

// Fundraising Readiness
GET /api/fundraising/readiness
GET /api/fundraising/investor-matches
POST /api/fundraising/one-pager

// Alerts
GET /api/alerts
PUT /api/alerts/:id/acknowledge
DELETE /api/alerts/:id
GET /api/alerts/rules
```

```
POST /api/alerts/rules
PUT /api/alerts/rules/:id
DELETE /api/alerts/rules/:id

// Slack Integration
GET /api/integrations/slack/auth
POST /api/integrations/slack/callback
POST /api/integrations/slack/notify
```

📁 FILE STRUCTURE

```
app/
├── (dashboard)/
│   ├── layout.tsx
│   ├── page.tsx (existing dashboard)
│   ├── scenarios/
│   ├── decisions/
│   ├── fundraising/
│   │   ├── page.tsx
│   │   └── components/
│   ├── hiring-planner/
│   │   ├── page.tsx
│   │   └── components/
│   ├── alerts/
│   │   ├── page.tsx
│   │   └── components/
│   └── journal/
├── components/
│   ├── board-export/
│   ├── ui/ (shadcn components)
│   └── layout/
├── hooks/
│   ├── useHiringPlan.ts
│   ├── useAlerts.ts
│   ├── useReadiness.ts
│   └── useBoardExport.ts
├── lib/
│   ├── utils.ts
│   ├── readinessScoring.ts
│   ├── alertEngine.ts
│   ├── anomalyDetection.ts
│   └── exportGenerators/
│       ├── pdfGenerator.ts
│       ├── slidesGenerator.ts
│       └── narrativeGenerator.ts
├── api/
│   ├── export/
│   ├── hiring-plans/
│   ├── alerts/
```

```
│   ├── fundraising/
│   └── integrations/
├── data/
│   ├── roles.ts
│   ├── investors.ts
│   └── templates/
└── types/
    ├── hiring.ts
    ├── alerts.ts
    ├── fundraising.ts
    └── export.ts
components/ui/ (shadcn/ui components)
public/
  └── templates/ (export templates)
```

✅ IMPLEMENTATION CHECKLIST

Phase 1: Core Infrastructure (Week 1)

- Set up database schema
- Create TypeScript types/interfaces
- Install dependencies
- Create API route stubs

Phase 2: Board Export (Week 1-2)

- Build ExportButton component
- Create template system
- Implement PDF generator
- Add AI narrative generation
- Test exports with real data

Phase 3: Hiring Planner (Week 2-3)

- Create role library data
- Build RoleSelector component
- Build HiringTimeline component
- Implement cost calculations
- Add runway impact visualization
- Integrate with simulation engine

Phase 4: Fundraising Readiness (Week 3-4)

- Implement scoring algorithm
- Build ReadinessScore UI
- Create recommendations engine
- Add investor matching
- Build one-pager generator

Phase 5: Smart Alerts (Week 4-5)

- Build alert detection engine
- Create AlertCenter UI
- Implement email notifications
- Add Slack OAuth integration
- Build weekly briefing email

Phase 6: Polish & Testing (Week 5-6)

- Add loading states
- Error handling
- Responsive design
- Accessibility (ARIA labels)
- End-to-end testing

🎯 SUCCESS CRITERIA

- Board export generates professional PDF in <5 seconds
- Hiring planner shows accurate runway impact within 1% of simulation
- Readiness score updates in real-time as metrics change
- Alerts trigger within 1 hour of threshold breach
- All features work on mobile (responsive)
- 100% TypeScript coverage (no `any` types)

Build these features with production-quality code, following React best practices, proper error handling, and the existing codebase patterns. Focus on user experience — every feature should save founders time and give them confidence in their decisions.